Linux for Beginners

A Practical Guide

Chapter 8 of 12

Networking Basics: Connections, IPs & SSH

Contents

Chapter 8

Networking Basics: Connections, IPs & SSH

8.1 How Linux Networking Works

Linux is built for networking. The majority of the world's servers, cloud infrastructure, and internet backbone runs on Linux — and understanding how Linux handles network connections gives you insight into how the internet itself works.

At its core, Linux networking follows the same layered model as all modern networking: physical hardware (a network card or Wi-Fi adapter) connects to a network, gets an IP address, and communicates using protocols like TCP and UDP. Linux exposes all of this through the filesystem — network interfaces appear as named devices, and everything is configurable from the command line.

Key networking concepts

Concept	What It Means
IP address	A unique numerical label identifying a device on a network (e.g. 192.168.1.5)
Subnet mask	Defines which part of an IP is the network and which is the host
Gateway	The router address — where traffic goes when leaving the local network
DNS	Domain Name System — translates names like google.com to IP addresses
Port	A numbered endpoint on a host — e.g. port 22 (SSH), 80 (HTTP), 443 (HTTPS)
Interface	A network device: eth0 (wired), wlan0 (wireless), lo (loopback)
TCP	Connection-oriented protocol — reliable, ordered delivery (web, SSH, email)
UDP	Connectionless protocol — fast but no guarantee of delivery (DNS, video)

8.2 Network Interfaces and IP Addresses

A **network interface** is how Linux represents a network connection. Each physical or virtual network adapter has an interface name. The primary tool for inspecting and configuring interfaces is the `ip` command.

```bash
# Show all network interfaces and their IP addresses:
$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    link/ether 52:54:00:ab:cd:ef brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
3: wlan0: <BROADCAST,MULTICAST> mtu 1500
    link/ether aa:bb:cc:dd:ee:ff brd ff:ff:ff:ff:ff:ff

# Shorter alias:
$ ip a

# Show routing table:
$ ip route show
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100
```

Common interface names

Interface	What It Represents
lo	Loopback — always 127.0.0.1 — for local communication within the machine
eth0 / ens3	Wired Ethernet — naming varies by system (older: eth0, newer: ens3, enp2s0)
wlan0 / wlp	Wireless (Wi-Fi) interface
docker0	Virtual bridge created by Docker for container networking
tun0 / tap0	Virtual tunnel interfaces — used by VPNs

8.3 Testing Connectivity with ping

`ping` sends ICMP echo requests to a host and measures the round-trip time. It's the first tool to reach for when diagnosing network problems.

```bash
# Ping a host (runs forever — Ctrl+C to stop):
$ ping google.com
PING google.com (142.250.80.46) 56(84) bytes of data.
64 bytes from 142.250.80.46: icmp_seq=1 ttl=118 time=12.4 ms
64 bytes from 142.250.80.46: icmp_seq=2 ttl=118 time=11.9 ms
^C
--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss
rtt min/avg/max/mdev = 11.9/12.1/12.4/0.2 ms

# Ping only 4 times:
$ ping -c 4 google.com

# Ping an IP directly (bypasses DNS):
$ ping 8.8.8.8

# Diagnose step by step:
$ ping 127.0.0.1       # test loopback (is networking up at all?)
$ ping 192.168.1.1     # test gateway (can you reach the router?)
$ ping 8.8.8.8         # test internet (can you reach the internet?)
$ ping google.com      # test DNS (can you resolve names?)
```

8.4 Downloading Files: curl and wget

Two essential tools for fetching data from the internet directly in the terminal. Both are widely used in scripts, automation, and everyday Linux work.

curl — transfer data to/from URLs

```bash
# Download a file:
$ curl -O https://example.com/file.tar.gz

# Download and save with a custom name:
$ curl -o myfile.tar.gz https://example.com/file.tar.gz

# Follow redirects (important for many URLs):
$ curl -L https://example.com/redirect

# Show HTTP headers only:
$ curl -I https://example.com

# Send a POST request with JSON:
$ curl -X POST -H 'Content-Type: application/json' \
    -d '{"name": "ashik"}' https://api.example.com/users

# Test an API endpoint:
$ curl https://api.github.com/users/ashihh07
```

wget — download files and mirror websites

```bash
# Download a file:
$ wget https://example.com/file.tar.gz

# Download quietly (no progress output):
$ wget -q https://example.com/file.tar.gz

# Continue an interrupted download:
$ wget -c https://example.com/largefile.iso

# Download in background:
$ wget -b https://example.com/largefile.iso
```

Feature	curl	wget
Primary use	Transferring data (upload + download)	Downloading files
HTTP methods	Full support (GET, POST, PUT, DELETE)	GET only by default
API interaction	Excellent	Limited
Resume download	Partial support	Yes (-c flag)
Recursive download	No	Yes (--recursive)
Scripting	Very common	Common for file downloads

8.5 Inspecting Open Connections: ss and netstat

These tools show what network connections are open on your system — which ports are listening, which connections are established, and which processes own them.

```bash
# ss is the modern replacement for netstat:

# Show all listening ports:
$ ss -tlnp
State  Recv-Q Send-Q  Local Address:Port  Peer Address:Port  Process
LISTEN 0      128          0.0.0.0:22         0.0.0.0:*      users:(('sshd',pid=423))
LISTEN 0      511          0.0.0.0:80         0.0.0.0:*      users:(('nginx',pid=1423))

# Show all established connections:
$ ss -tnp

# Show UDP connections:
$ ss -unp

# Show all sockets (listening + connected):
$ ss -anp

# Legacy netstat (still available on many systems):
$ netstat -tlnp
```

Flag	Meaning
-t	TCP sockets
-u	UDP sockets
-l	Listening sockets only
-n	Show numbers not names (faster)
-p	Show process that owns the socket
-a	All sockets (listening and connected)

8.6 DNS: How Names Become IP Addresses

When you type `google.com`, Linux needs to find its IP address before it can connect. This lookup is handled by **DNS** (Domain Name System). Three tools let you query and debug DNS from the terminal.

```bash
# dig — the most detailed DNS lookup tool:
$ dig google.com
;; ANSWER SECTION:
google.com.  300  IN  A  142.250.80.46

# Look up a specific record type:
$ dig google.com MX       # mail servers
$ dig google.com NS       # nameservers
$ dig google.com AAAA     # IPv6 address

# Reverse lookup — IP to hostname:
$ dig -x 8.8.8.8

# nslookup — simpler DNS query:
$ nslookup google.com
Server:  127.0.0.53
Address: 127.0.0.53#53
Name:    google.com
Address: 142.250.80.46

# host — the quickest DNS check:
$ host google.com
google.com has address 142.250.80.46
google.com mail is handled by 10 smtp.google.com.

# View your DNS resolver config:
$ cat /etc/resolv.conf
```

8.7 SSH: Connecting to Remote Servers

SSH (Secure Shell) is how you connect to and control remote Linux servers. Every command you've learned in this guide can be used over SSH on a machine across the room or across the world. It's the most important networking tool in Linux.

SSH encrypts everything — your password, your commands, and any data transferred. Nothing travels in plaintext. This is what makes it safe to use over untrusted networks.

Basic SSH usage

```bash
# Connect to a remote server:
$ ssh username@hostname
$ ssh ashik@192.168.1.50
$ ssh ashik@server.example.com

# Connect on a non-standard port (default is 22):
$ ssh -p 2222 ashik@server.example.com

# Run a single command on the remote server without opening a shell:
$ ssh ashik@server.example.com 'df -h'

# Connect with verbose output (useful for debugging):
$ ssh -v ashik@server.example.com
```

What happens when you first connect

```bash
$ ssh ashik@192.168.1.50
The authenticity of host '192.168.1.50' can't be established.
ED25519 key fingerprint is SHA256:abc123...xyz789.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.50' (ED25519) to known hosts.
ashik@192.168.1.50's password:
```

The first time you connect to a server, SSH shows its fingerprint and asks you to verify it. Type `yes` to accept — the fingerprint is saved to `~/.ssh/known_hosts`. On future connections, SSH silently verifies the server hasn't changed. If the fingerprint changes unexpectedly, SSH will warn you — a potential sign of a man-in-the-middle attack.

8.8 SSH Keys: Passwordless and Secure Authentication

Typing a password every time you SSH is slow and less secure than using **SSH keys**. A key pair consists of a **private key** (stays on your machine, never shared) and a **public key** (placed on the server). Authentication happens cryptographically — no password is ever sent over the network.

Generating an SSH key pair

```bash
# Generate a new ED25519 key pair (recommended — modern and fast):
$ ssh-keygen -t ed25519 -C 'ashik@mycomputer'
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ashik/.ssh/id_ed25519): [Enter]
Enter passphrase (empty for no passphrase): [Enter or type passphrase]
Your identification has been saved in /home/ashik/.ssh/id_ed25519
Your public key has been saved in /home/ashik/.ssh/id_ed25519.pub

# View your public key (this is what you share):
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAA...longstring... ashik@mycomputer
```

Copying your public key to a server

```bash
# The easiest way — ssh-copy-id does it automatically:
$ ssh-copy-id ashik@192.168.1.50
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed.
ashik@192.168.1.50's password:
Number of key(s) added: 1

# After this, you can SSH without a password:
$ ssh ashik@192.168.1.50
# Logs in immediately — no password prompt

# Manual method (if ssh-copy-id isn't available):
$ cat ~/.ssh/id_ed25519.pub | ssh ashik@server 'mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys
```

SSH key file permissions

```bash
# SSH is strict about key file permissions:
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/id_ed25519      # private key — only you can read
$ chmod 644 ~/.ssh/id_ed25519.pub  # public key — readable by others
$ chmod 600 ~/.ssh/authorized_keys
```

WARNING
Always set a passphrase on your private key. If someone steals your private key file without a passphrase, they have immediate access to every server it's authorised on. Use ssh-agent to avoid typing the passphrase repeatedly.

8.9 The SSH Config File

Typing `ssh ashik@192.168.1.50 -p 2222 -i ~/.ssh/special_key` every time is tedious. The SSH config file lets you define shortcuts for your connections.

```bash
$ nano ~/.ssh/config

# Add entries like this:
Host myserver
    HostName 192.168.1.50
    User ashik
    Port 2222
    IdentityFile ~/.ssh/id_ed25519

Host github
    HostName github.com
    User git
    IdentityFile ~/.ssh/github_key

Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
```

```bash
# Now connect with just:
$ ssh myserver
# Instead of: ssh ashik@192.168.1.50 -p 2222 -i ~/.ssh/id_ed25519

# Set correct permissions on the config file:
$ chmod 600 ~/.ssh/config
```

> **TIP**
> The Host * block at the bottom applies to all connections. ServerAliveInterval 60 sends a keepalive every 60 seconds so your SSH sessions don't drop due to inactivity.

8.10 Transferring Files: scp and rsync

SSH isn't just for interactive shells — it's also the transport layer for secure file transfers.

scp — secure copy

```bash
# Copy a local file TO a remote server:
$ scp file.txt ashik@server:/home/ashik/

# Copy a file FROM a remote server:
$ scp ashik@server:/home/ashik/file.txt .

# Copy a directory recursively:
$ scp -r my_project/ ashik@server:/home/ashik/

# Use a non-standard port:
$ scp -P 2222 file.txt ashik@server:/tmp/
```

rsync — efficient sync with delta transfers

`rsync` only transfers the parts of files that have changed. For large files or frequent syncs, it's dramatically faster than `scp`.

```bash
# Sync a directory to a remote server:
$ rsync -avz my_project/ ashik@server:/home/ashik/my_project/

# Sync from remote to local:
$ rsync -avz ashik@server:/var/log/ ./server_logs/

# Dry run — show what would be transferred without doing it:
$ rsync -avzn my_project/ ashik@server:/home/ashik/my_project/

# Delete files on destination that no longer exist on source:
$ rsync -avz --delete my_project/ ashik@server:/home/ashik/my_project/
```

Flag	Meaning
-a	Archive mode — preserves permissions, timestamps, symlinks, recursion
-v	Verbose — show files being transferred
-z	Compress during transfer — faster over slow connections
-n	Dry run — simulate without making changes
--delete	Remove destination files that no longer exist in source
--progress	Show transfer progress per file

8.11 SSH Tunnels and Port Forwarding

SSH can forward network ports through an encrypted tunnel — a powerful feature for accessing services on remote machines as if they were local, or for securely exposing local services.

Local port forwarding

Makes a port on the remote server accessible as a local port on your machine:

```bash
# Access a remote database (port 5432) as localhost:5432:
$ ssh -L 5432:localhost:5432 ashik@server

# Syntax: -L local_port:remote_host:remote_port
# Now connect locally:
$ psql -h localhost -p 5432 -U dbuser mydb
```

Remote port forwarding

Exposes a local port on the remote server — useful for sharing a local dev server:

```bash
# Expose your local web server (port 3000) on the remote server's port 8080:
$ ssh -R 8080:localhost:3000 ashik@server
# Syntax: -R remote_port:local_host:local_port
```

8.12 Firewalls with ufw

Linux includes a powerful firewall called **iptables** (and the newer nftables), but configuring it directly is complex. **ufw** (Uncomplicated Firewall) is a beginner-friendly frontend that Ubuntu installs by default.

```bash
# Check firewall status:
$ sudo ufw status
Status: inactive

# Enable the firewall:
$ sudo ufw enable

# Allow SSH (do this BEFORE enabling if connecting remotely!):
$ sudo ufw allow ssh
$ sudo ufw allow 22

# Allow common services:
$ sudo ufw allow http      # port 80
$ sudo ufw allow https     # port 443
$ sudo ufw allow 3000      # custom port

# Deny a port:
$ sudo ufw deny 8080

# Delete a rule:
$ sudo ufw delete allow 3000

# Check status with rule numbers:
$ sudo ufw status numbered

# Allow from a specific IP only:
$ sudo ufw allow from 192.168.1.50 to any port 22
```

WARNING
Always allow SSH (sudo ufw allow ssh) BEFORE enabling the firewall on a remote server. Enabling ufw without an SSH rule will lock you out immediately.

8.13 Practical Networking Scenarios

Scenario 1: Diagnose no internet connection

```bash
# Step 1: is the interface up?
$ ip addr show eth0

# Step 2: can you reach the router?
$ ping -c 3 192.168.1.1

# Step 3: can you reach the internet by IP?
$ ping -c 3 8.8.8.8

# Step 4: is DNS working?
$ ping -c 3 google.com
$ dig google.com
```

Scenario 2: Set up SSH key-based login on a new server

```bash
# On your local machine:
$ ssh-keygen -t ed25519 -C 'mykey'
$ ssh-copy-id user@newserver

# Verify it works:
$ ssh user@newserver

# On the server — optionally disable password auth:
$ sudo nano /etc/ssh/sshd_config
# Set: PasswordAuthentication no
$ sudo systemctl restart sshd
```

Scenario 3: Check what's listening on a port

```bash
# Is anything listening on port 80?
$ ss -tlnp | grep :80
LISTEN 0  511  0.0.0.0:80  0.0.0.0:*  users:(("nginx",pid=1423,fd=6))

# What process is using port 3000?
$ ss -tlnp | grep :3000
```

Scenario 4: Sync a local project to a server

```bash
# Dry run first to check what will be sent:
$ rsync -avzn ~/my_project/ ashik@server:/var/www/my_project/

# If the dry run looks correct, run for real:
$ rsync -avz ~/my_project/ ashik@server:/var/www/my_project/
```

8.14 Essential Commands: Quick Reference

Command	What It Does	Common Usage
ip addr show	Show network interfaces and IPs	ip a
ip route show	Show routing table and gateway	ip route show
ping host	Test connectivity to a host	ping -c 4 google.com
curl -O URL	Download a file	curl -O https://...
curl -I URL	Show HTTP response headers only	curl -I https://example.com
wget URL	Download a file	wget https://...
ss -tlnp	Show listening TCP ports and processes	ss -tlnp
ss -anp	Show all sockets	ss -anp
dig domain	DNS lookup with full details	dig google.com MX
nslookup domain	Simple DNS lookup	nslookup google.com
host domain	Quick DNS lookup	host google.com
ssh user@host	Connect to a remote server	ssh ashik@192.168.1.50
ssh -p PORT user@host	SSH on a non-standard port	ssh -p 2222 ashik@server
ssh-keygen -t ed25519	Generate an SSH key pair	ssh-keygen -t ed25519
ssh-copy-id user@host	Copy public key to server	ssh-copy-id ashik@server
scp file user@host:path	Copy file to remote server	scp app.py ashik@server:~/
scp user@host:path .	Copy file from remote server	scp ashik@server:~/log.txt .
rsync -avz src dst	Sync files efficiently over SSH	rsync -avz project/ server:~/
ssh -L lport:host:rport	Local port forwarding tunnel	ssh -L 5432:localhost:5432 server
sudo ufw status	Check firewall status	sudo ufw status
sudo ufw allow ssh	Allow SSH through firewall	sudo ufw allow ssh
sudo ufw enable	Enable the firewall	sudo ufw enable

8.15 Chapter Summary

✓ Linux networking is built around **interfaces** (eth0, wlan0, lo), **IP addresses**, **routes**, and **DNS**. The `ip` command is the primary tool for inspecting and configuring all of these.

✓ `ping` is the first diagnostic tool — test loopback, then gateway, then internet IP, then DNS to isolate any connectivity problem.

✓ `curl` transfers data and interacts with APIs. `wget` downloads files and supports resuming interrupted downloads.

✓ `ss -tlnp` shows what's listening on which ports and which process owns each socket. Use it to confirm services are running and to find port conflicts.

✓ SSH is the foundation of remote Linux administration — encrypted, reliable, and available everywhere. Learn it thoroughly.

✓ **SSH keys** are more secure than passwords and faster to use. Always use ED25519, always set a passphrase, always set correct permissions (`chmod 600` on private keys).

✓ The `~/.ssh/config` file lets you define shortcuts for all your SSH connections — host aliases, ports, users, and key files in one place.

✓ `scp` copies files over SSH simply. `rsync` syncs directories efficiently — only transferring what changed. Always do a dry run (`-n`) before a real rsync.

✓ SSH tunnels forward ports through encrypted connections — useful for accessing remote databases, services, and APIs as if they were local.

✓ `ufw` makes firewall management approachable. Always allow SSH before enabling the firewall on a remote server.

What's Coming in Chapter 9

With networking mastered, Chapter 9 brings everything together: **shell scripting**. You'll learn to automate tasks by combining everything you've learned into reusable scripts:

- What a shell script is and how the shebang line (`#!/bin/bash`) works
- Variables, user input, and command substitution
- Conditionals: `if`, `elif`, `else`, and test expressions
- Loops: `for`, `while`, and `until`
- Functions — reusable blocks of logic
- Reading arguments, handling errors, and exit codes
- Practical scripts: backups, log rotation, system health checks

Chapter 9 is where Linux stops being a tool you use and starts being a tool you program.

Linux for Beginners — Chapter 8 of 12

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.